

SISTEMA DE GESTÃO IMOBILIÁRIA PARA CORRETORES – CORRETOR PRO
DOCUMENTAÇÃO DO PROJETO

Instituição

UNIGOIÁS – Centro Universitário de Goiás

Curso

Análise e Desenvolvimento de Sistemas

Disciplina

Projeto Final ADS

Professor

Marcio de Souza Balian

Aluno

Maycon Cordeiro

Cidade

Goiânia – GO

Ano

2026

SUMÁRIO

1. Introdução
2. Objetivos
3. Justificativa
4. Escopo do Projeto
5. Requisitos Funcionais
6. Requisitos Não Funcionais
7. Diagrama de Casos de Uso
8. Diagrama de Classes
9. Arquitetura do Sistema
10. Banco de Dados
11. Modelo Entidade-Relacionamento (DER)
12. AJAX
13. Spring Framework e Spring Boot
14. Spring Security
15. JPA e Hibernate
16. JasperReports
17. Docker
18. Deploy em Nuvem (Render e Vercel)
19. Demonstração do Sistema
20. Conclusão
21. Referências

1. INTRODUÇÃO

O mercado imobiliário exige cada vez mais organização e controle das informações relacionadas aos imóveis, clientes e negociações. Muitos corretores ainda utilizam planilhas, anotações em papel ou aplicativos genéricos para controlar sua carteira de imóveis, o que pode ocasionar perda de informações, retrabalho e dificuldade na geração de relatórios.

Com o objetivo de solucionar esse problema, foi desenvolvido o sistema Corretor Pro, uma aplicação web destinada ao gerenciamento de imóveis e propostas comerciais. O sistema permite que corretores realizem o cadastro de imóveis, acompanhem propostas recebidas, gerem relatórios profissionais em formato PDF e mantenham todas as informações centralizadas em um único ambiente.

O projeto foi desenvolvido utilizando tecnologias modernas amplamente utilizadas no mercado de trabalho, como Spring Boot, Spring Security, JPA, Hibernate, PostgreSQL, AJAX, Docker, Render e Vercel.

Além disso, foram implementados mecanismos de autenticação e segurança para garantir que apenas usuários autorizados possam acessar as funcionalidades do sistema.

O desenvolvimento deste projeto possibilitou a aplicação prática dos conhecimentos adquiridos ao longo do curso de Análise e Desenvolvimento de Sistemas, contemplando conceitos de programação web, banco de dados, segurança da informação, arquitetura de software e computação em nuvem.

Tecnologias Utilizadas

- Java 17
- Spring Boot
- Spring Security
- JPA
- Hibernate
- PostgreSQL
- AJAX
- JasperReports
- Docker
- GitHub
- Render
- Vercel
- HTML5
- CSS3
- JavaScript

2. OBJETIVOS

Objetivo Geral

- Desenvolver uma aplicação web para gerenciamento imobiliário, permitindo o cadastro e administração de imóveis e propostas comerciais de forma segura, organizada e eficiente.

Objetivos Específicos

- Desenvolver uma interface web responsiva.
- Implementar autenticação de usuários.
- Permitir cadastro de novos usuários.
- Realizar cadastro, edição e exclusão de imóveis.
- Registrar propostas de compra.
- Excluir propostas cadastradas.
- Gerar relatórios em PDF.
- Persistir informações em banco de dados PostgreSQL.
- Aplicar conceitos de Spring Security.
- Aplicar conceitos de JPA e Hibernate.
- Utilizar AJAX para comunicação assíncrona.
- Disponibilizar a aplicação em ambiente de nuvem.

3. JUSTIFICATIVA

O gerenciamento de imóveis e propostas é uma atividade essencial para corretores imobiliários. A utilização de sistemas informatizados reduz erros operacionais, melhora a organização das informações e aumenta a produtividade do profissional.

O sistema Corretor Pro foi desenvolvido para demonstrar a aplicação prática dos conhecimentos adquiridos durante o curso, além de oferecer uma solução funcional para controle imobiliário.

O projeto também proporciona experiência com tecnologias amplamente utilizadas no mercado, contribuindo para a formação profissional do desenvolvedor.

4. ESCOPO DO PROJETO

O sistema Corretor Pro foi desenvolvido para auxiliar corretores de imóveis no gerenciamento de imóveis e propostas comerciais.

A aplicação disponibiliza funcionalidades de autenticação de usuários, cadastro e gerenciamento de imóveis, registro de propostas e emissão de relatórios em PDF.

O sistema contempla as seguintes áreas:

Usuários

- Cadastro de usuários.
- Autenticação através de login.
- Consulta de informações do usuário autenticado.

Imóveis

- Cadastro de imóveis.
- Consulta de imóveis cadastrados.
- Atualização de imóveis.
- Exclusão de imóveis.

Propostas

- Cadastro de propostas vinculadas aos imóveis.
- Consulta de propostas recebidas.
- Exclusão de propostas.
- Alteração do status da proposta (Aceita, Recusada ou Em Análise).

Relatórios

- Geração de relatórios em formato PDF.
- Relatório de imóveis cadastrados.
- Relatório de propostas cadastradas.

5. REQUISITOS FUNCIONAIS

Os requisitos funcionais representam as funcionalidades que o sistema deve executar.

RF01 – Cadastro de Usuário

O sistema deve permitir o cadastro de novos usuários.

RF02 – Login

O sistema deve permitir a autenticação dos usuários através de e-mail e senha.

RF03 – Cadastro de Imóvel

O sistema deve permitir o cadastro de imóveis.

RF04 – Consulta de Imóveis

O sistema deve permitir a visualização dos imóveis cadastrados.

RF05 – Atualização de Imóveis

O sistema deve permitir a edição dos dados dos imóveis cadastrados.

RF06 – Exclusão de Imóveis

O sistema deve permitir a exclusão de imóveis.

RF07 – Cadastro de Propostas

O sistema deve permitir o registro de propostas relacionadas aos imóveis.

RF08 – Consulta de Propostas

O sistema deve permitir a visualização das propostas cadastradas.

RF09 – Alteração de Status das Propostas

O sistema deve permitir alterar o status das propostas para:

- Em análise

- Aceita
- Recusada

RF10 – Exclusão de Propostas

O sistema deve permitir a exclusão de propostas.

RF11 – Geração de Relatórios PDF

O sistema deve permitir a emissão de relatórios em formato PDF.

6. REQUISITOS NÃO FUNCIONAIS

Os requisitos não funcionais definem as características técnicas e operacionais da aplicação.

RNF01 – Linguagem de Programação

O sistema deve ser desenvolvido utilizando Java 17.

RNF02 – Framework Backend

O sistema deve utilizar Spring Boot.

RNF03 – Segurança

O sistema deve utilizar Spring Security com autenticação baseada em JWT.

RNF04 – Persistência

O sistema deve utilizar JPA e Hibernate para persistência dos dados.

RNF05 – Banco de Dados

O sistema deve utilizar PostgreSQL.

RNF06 – Comunicação

O frontend deve utilizar AJAX para comunicação assíncrona com a API.

RNF07 – Relatórios

Os relatórios devem ser gerados em formato PDF utilizando JasperReports.

RNF08 – Containerização

O sistema deve possuir Dockerfile e Docker Compose para execução em containers.

RNF09 – Disponibilidade

O sistema deve permitir implantação em ambiente de nuvem.

RNF10 – Interface

O sistema deve possuir interface web responsiva para utilização em computadores e dispositivos móveis.

7. DIAGRAMA DE CASOS DE USO

Ator Principal:

CORRETOR

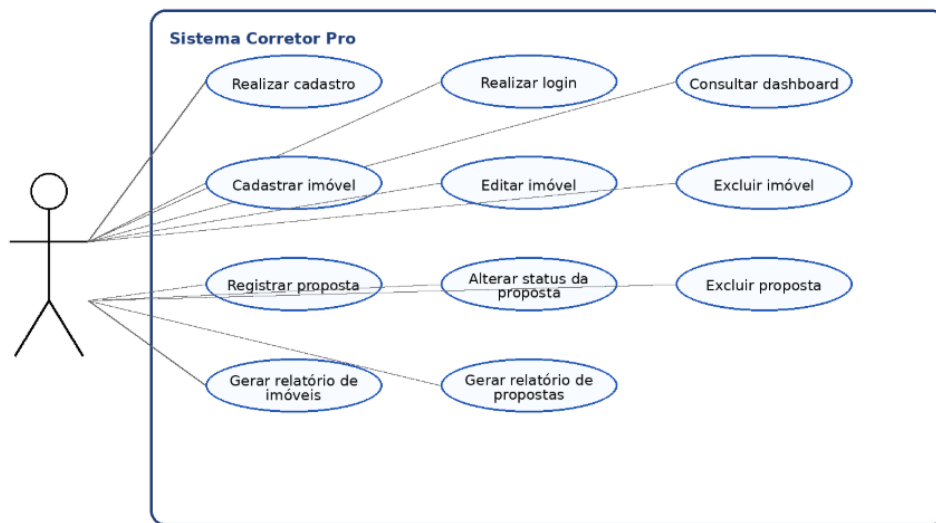
Casos de Uso:

- Realizar Cadastro
- Realizar Login
- Consultar Dashboard
- Cadastrar Imóvel
- Editar Imóvel
- Excluir Imóvel
- Consultar Imóveis
- Registrar Proposta
- Consultar Propostas
- Excluir Proposta
- Alterar Status da Proposta
- Gerar Relatório PDF

Relacionamentos:

O ator Corretor é responsável por executar todas as funcionalidades do sistema após autenticação.

UML - Diagrama Geral de Casos de Uso



8. DIAGRAMA DE CLASSES

Classe Usuario

Atributos:

- id
- nome
- email
- Senha

Classe Imovel

Atributos:

- id
- titulo
- endereco

- bairro
- cidade
- estado
- cep
- preco
- status
- imagem
- mapa

Classe Proposta

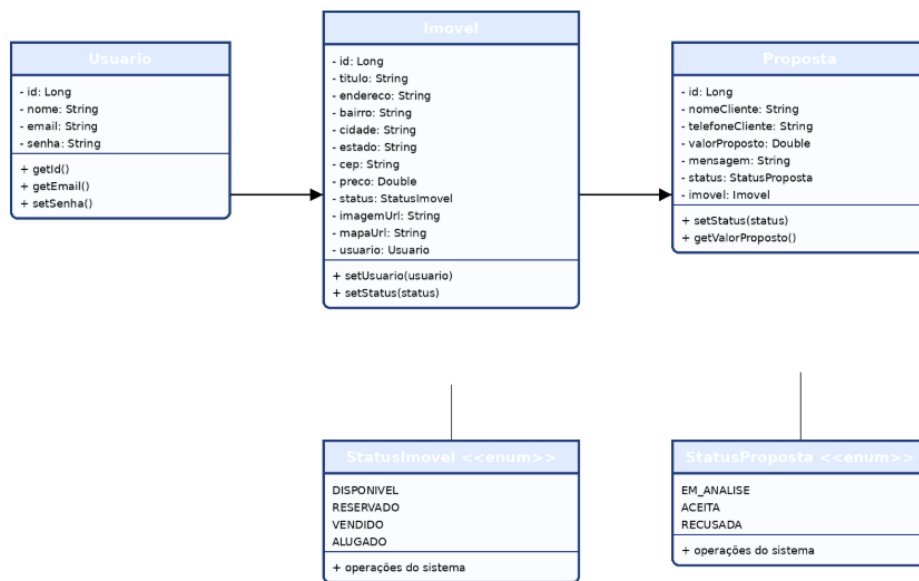
Atributos:

- id
- cliente
- telefone
- valor
- observacao
- status

Relacionamentos:

Usuario → gerencia → Imovel

Imovel → recebe → Proposta



9. ARQUITETURA DO SISTEMA

A arquitetura segue o modelo cliente-servidor.

Frontend:

- HTML5
- CSS3
- JavaScript

Backend:

- Spring Boot
- Spring Security
- JPA
- Hibernate

Banco:

- PostgreSQL

Hospedagem:

- Render
- Vercel

Fluxo:

Usuário → Frontend → API REST → Banco PostgreSQL

10. BANCO DE DADOS

O sistema utiliza PostgreSQL.

Tabelas principais:

usuario

- id
- nome
- email
- senha

imovel

- id
- titulo
- endereco
- bairro
- cidade
- estado
- cep
- preco
- status
- imagem
- mapa

proposta

- id
- cliente
- telefone
- valor
- observacao
- status

A criação e o gerenciamento das tabelas foram realizados através do JPA/Hibernate, que efetuou o mapeamento objeto-relacional entre as entidades Java e as tabelas do banco PostgreSQL.

11. MODELO ENTIDADE-RELACIONAMENTO (DER)

O Modelo Entidade-Relacionamento representa a estrutura lógica do banco de dados utilizado pelo sistema Corretor Pro. Ele demonstra as principais entidades persistidas no PostgreSQL e os relacionamentos existentes entre elas.

As principais entidades do sistema são: Usuario, Imovel e Proposta.

A entidade Usuario armazena os dados dos usuários cadastrados no sistema, como nome, e-mail e senha.

A entidade Imovel representa os imóveis cadastrados pelo corretor, contendo informações como título,

endereço, bairro, cidade, estado, CEP, preço, status, imagem e mapa. A entidade Proposta representa as propostas recebidas para os imóveis cadastrados, armazenando dados do cliente, telefone, valor proposto, mensagem e status.

Estrutura das entidades:

USUARIO

- id
- nome
- email
- senha

IMOVEL

- id
- titulo
- endereco
- bairro
- cidade
- estado
- cep
- preco
- status
- imagem_url
- mapa_url
- usuario_id

PROPOSTA

- id
- nome_cliente
- telefone_cliente
- valor_proposto
- mensagem
- status
- imovel_id

Relacionamentos:

USUARIO 1 ---- N IMOVEL

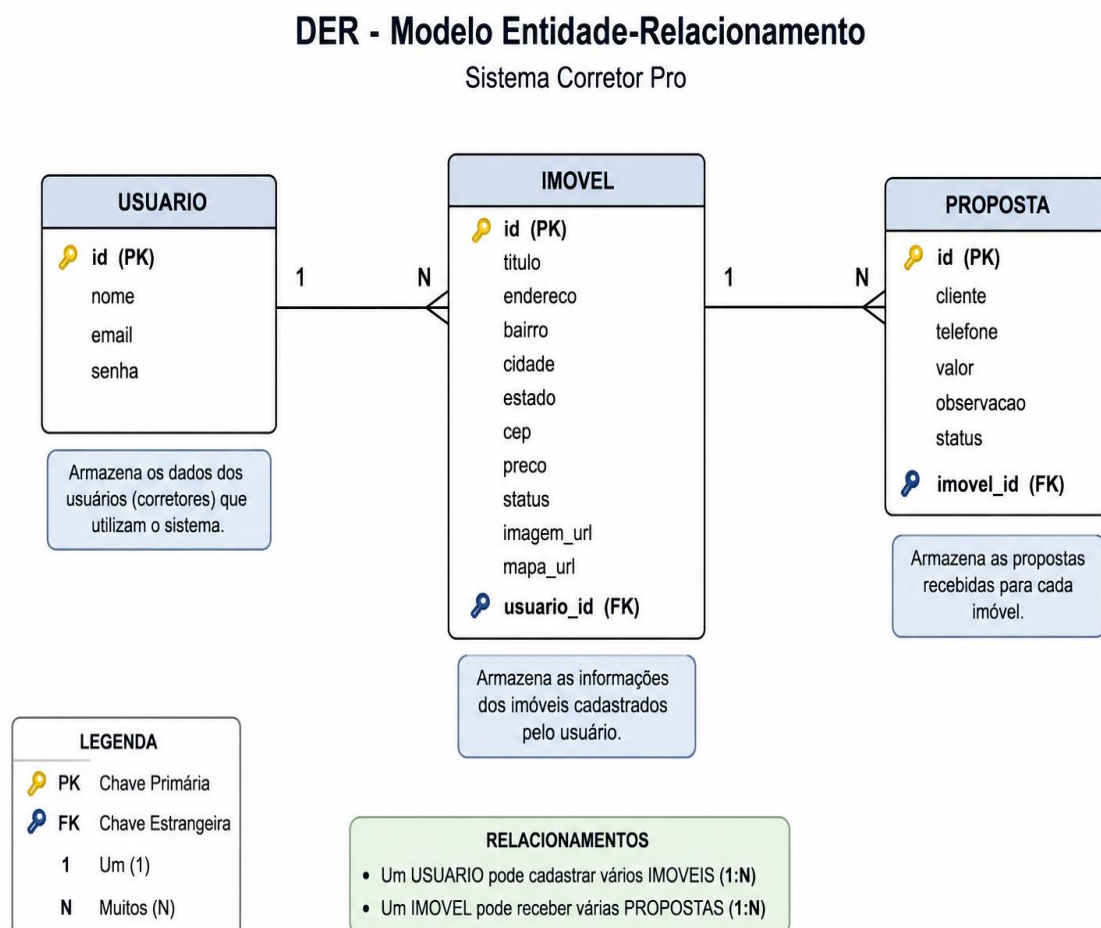
Um usuário pode cadastrar vários imóveis, porém cada imóvel pertence a apenas um usuário.

IMOVEL 1 ---- N PROPOSTA

Um imóvel pode receber várias propostas, porém cada proposta pertence a apenas um imóvel.

Representação simplificada do DER:

MODELO ENTIDADE-RELACIONAMENTO (DER)



O Modelo Entidade-Relacionamento representa a estrutura lógica do banco de dados utilizado pelo sistema Corretor Pro.

A entidade Usuario armazena os dados dos usuários cadastrados e autenticados no sistema.

A entidade Imovel armazena as informações dos imóveis cadastrados pelos corretores, incluindo localização, preço, status e imagens.

A entidade Proposta registra as propostas realizadas para os imóveis cadastrados.

O relacionamento entre Usuario e Imovel é do tipo um para muitos (1), permitindo que um usuário possua vários imóveis cadastrados.

O relacionamento entre Imovel e Proposta também é do tipo um para muitos (1), permitindo que um imóvel receba diversas propostas.

12. AJAX

AJAX foi utilizado para realizar chamadas assíncronas entre o frontend e a API desenvolvida em Spring Boot.

Exemplos:

- Cadastro de usuários
- Login
- Cadastro de imóveis
- Consulta de imóveis
- Cadastro de propostas
- Exclusão de registros
- Geração de relatórios

A comunicação ocorre utilizando a API Fetch do JavaScript.

13. SPRING FRAMEWORK E SPRING BOOT

- Spring Boot é o framework principal utilizado no backend.

- Responsabilidades:
- Inicialização da aplicação
- Criação dos endpoints REST
- Injeção de dependências
- Integração com banco de dados
- Configuração simplificada

Exemplo:

```
</>JavaScript
```

```
@RestController
```

```
@RequestMapping("/imoveis")
```

14. SPRING SECURITY

- O Spring Security foi utilizado para proteger a aplicação.
- Funcionalidades:
 - Controle de autenticação
 - Proteção de rotas
 - Criptografia de senhas
 - Integração com JWT
- Benefícios:
 - Segurança dos dados
 - Controle de acesso
 - Proteção contra acessos não autorizados

Exemplo:

```
</>JavaScript
```

.anyRequest().authenticated()

A autenticação foi implementada utilizando JSON Web Token (JWT), permitindo que o usuário realize login e receba um token de acesso utilizado nas requisições protegidas da aplicação.

15. JPA e Hibernate

O projeto utiliza JPA (Java Persistence API) para mapeamento objeto-relacional e Hibernate como implementação do JPA. As entidades do sistema são mapeadas através da anotação `@Entity`, permitindo que as classes Java sejam convertidas em tabelas do banco PostgreSQL automaticamente.

Os repositórios utilizam a interface `JpaRepository`, responsável pelas operações de persistência, consulta, atualização e exclusão de registros.

```
src > main > java > com > corretor > corretor > model > J Usuario.java > {} com.corretor.corretor.model
1 package com.corretor.corretor.model;
2
3 import jakarta.persistence.*;
4 import com.fasterxml.jackson.annotation.JsonProperty;
5
6 @Entity
7 @Table(name = "usuario")
8 public class Usuario {
9
10     @Id
11     @GeneratedValue(strategy = GenerationType.IDENTITY)
12     private Long id;
13
14     private String nome;
15
16     @Column(unique = true)
17     private String email;
18
19     @JsonProperty(access = JsonProperty.Access.WRITE_ONLY)
20     private String senha;
21
22     // getters e setters
23     public Long getId() { return id; }
24     public void setId(Long id) { this.id = id; }
25
26     public String getNome() { return nome; }
27     public void setNome(String nome) { this.nome = nome; }
28
29     public String getEmail() { return email; }
30     public void setEmail(String email) { this.email = email; }
31
32     public String getSenha() { return senha; }
33     public void setSenha(String senha) { this.senha = senha; }
34 }
```

Imagem 1 - Entidade Usuario mapeada com JPA/Hibernate utilizando a anotação `@Entity`.

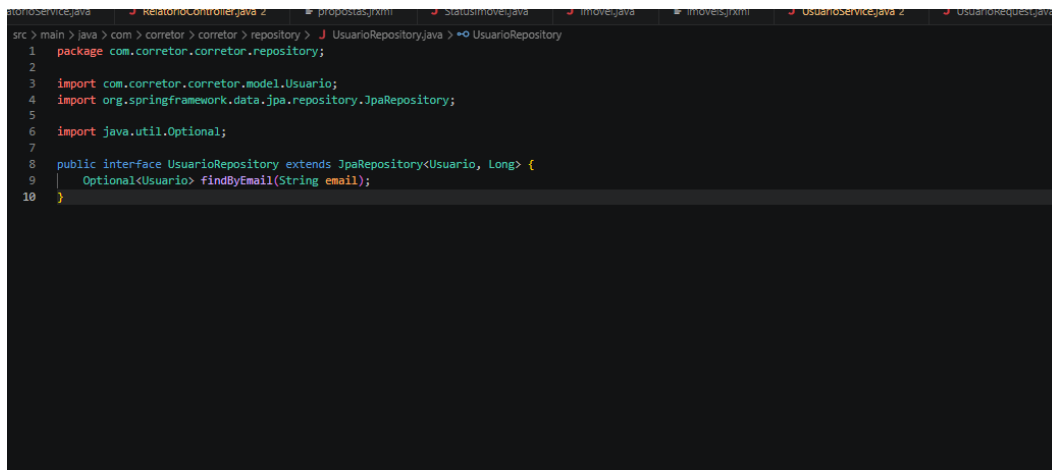


Imagem 2 - Repositório UsuarioRepository utilizando JpaRepository para persistência dos dados.

16. JASPERREPORTS

O JasperReports foi utilizado para geração dos relatórios PDF.

Relatórios implementados:

- Relatório de Imóveis
- Relatório de Propostas

Os relatórios são gerados dinamicamente utilizando os dados armazenados no banco de dados.

17. DOCKER

Docker foi utilizado para possibilitar a containerização da aplicação.

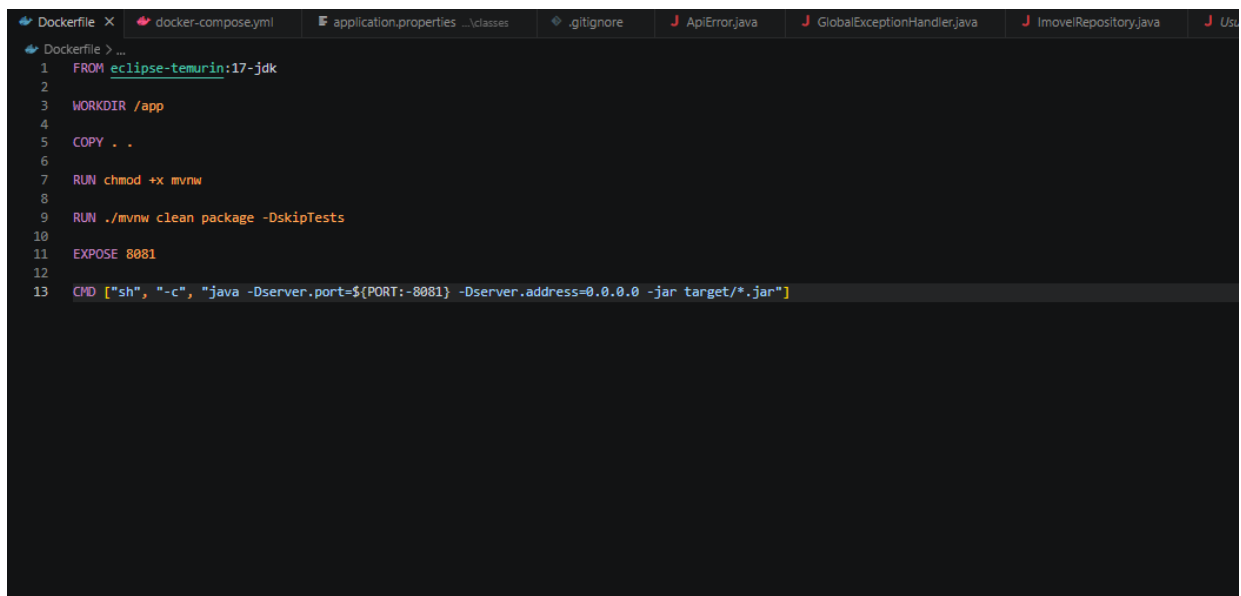
Benefícios:

- Padronização do ambiente
- Facilidade de deploy

- Portabilidade
- Escalabilidade

Arquivos utilizados:

- Dockerfile
- docker-compose.yml

A screenshot of a code editor showing a Dockerfile. The editor has a dark theme and a tab bar at the top with several files open: Dockerfile, docker-compose.yml, application.properties, .gitignore, ApiError.java, GlobalExceptionHandler.java, ImovelRepository.java, and Usu... The Dockerfile content is as follows:

```
1 FROM eclipse-temurin:17-jdk
2
3 WORKDIR /app
4
5 COPY . .
6
7 RUN chmod +x mvnw
8
9 RUN ./mvnw clean package -DskipTests
10
11 EXPOSE 8081
12
13 CMD ["sh", "-c", "java -Dserver.port=${PORT:-8081} -Dserver.address=0.0.0.0 -jar target/*.jar"]
```

O projeto possui um Dockerfile responsável pela construção da imagem da aplicação Spring Boot e um arquivo docker-compose.yml para orquestração dos serviços necessários durante a execução.

A utilização do Docker permite a padronização do ambiente de desenvolvimento e facilita futuras implantações em ambientes de homologação e produção.

18. DEPLOY EM NUVEM

Durante o desenvolvimento do projeto, foi realizada a preparação da aplicação para execução em ambiente de nuvem utilizando as plataformas Render e Vercel.

O frontend foi configurado para hospedagem na plataforma Vercel, permitindo acesso às páginas da aplicação através da internet.

O backend foi preparado para deploy na plataforma Render, utilizando uma API desenvolvida com Spring Boot e banco de dados PostgreSQL.

Por questões relacionadas à expiração do banco de dados gratuito disponibilizado pela plataforma Render, a versão apresentada para avaliação foi executada em ambiente local, utilizando Spring Boot, PostgreSQL e pgAdmin instalados no computador de desenvolvimento.

Dessa forma, todas as funcionalidades do sistema puderam ser demonstradas normalmente, incluindo:

- Cadastro de usuários;
- Autenticação (login);
- Cadastro de imóveis;
- Edição e exclusão de imóveis;
- Cadastro e exclusão de propostas;
- Geração de relatórios PDF;
- Persistência dos dados no PostgreSQL.

O sistema foi desenvolvido seguindo os requisitos propostos pela disciplina, utilizando Spring Boot, Spring Security, JPA/Hibernate, AJAX, PostgreSQL e JasperReports. Embora a aplicação tenha sido preparada para execução em nuvem através do Render e Vercel, a apresentação final foi realizada em ambiente local devido à indisponibilidade temporária do banco de dados gratuito da plataforma de hospedagem.

Render

Responsável pela hospedagem do backend Spring Boot.

Funções:

- Execução da API REST
- Comunicação com banco PostgreSQL
- Disponibilização dos endpoints

Vercel

Responsável pela hospedagem do frontend.

Funções:

- Hospedagem das páginas HTML
- Disponibilização da interface do usuário
- Integração com a API hospedada no Render

19. DEMONSTRAÇÃO DO SISTEMA

Inserir prints de:

- Tela Inicial

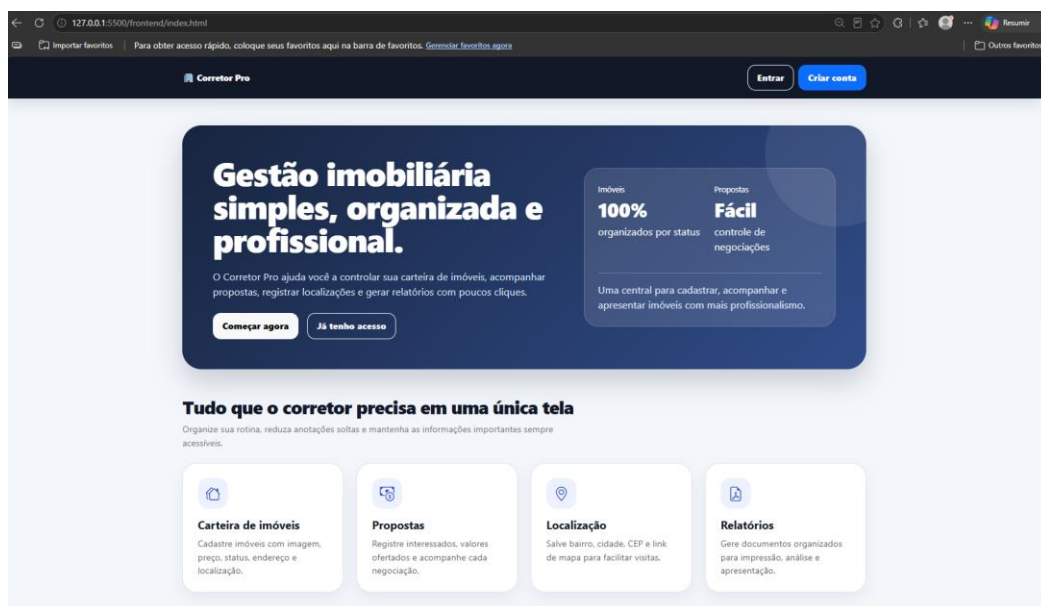


Imagem 3

- Login

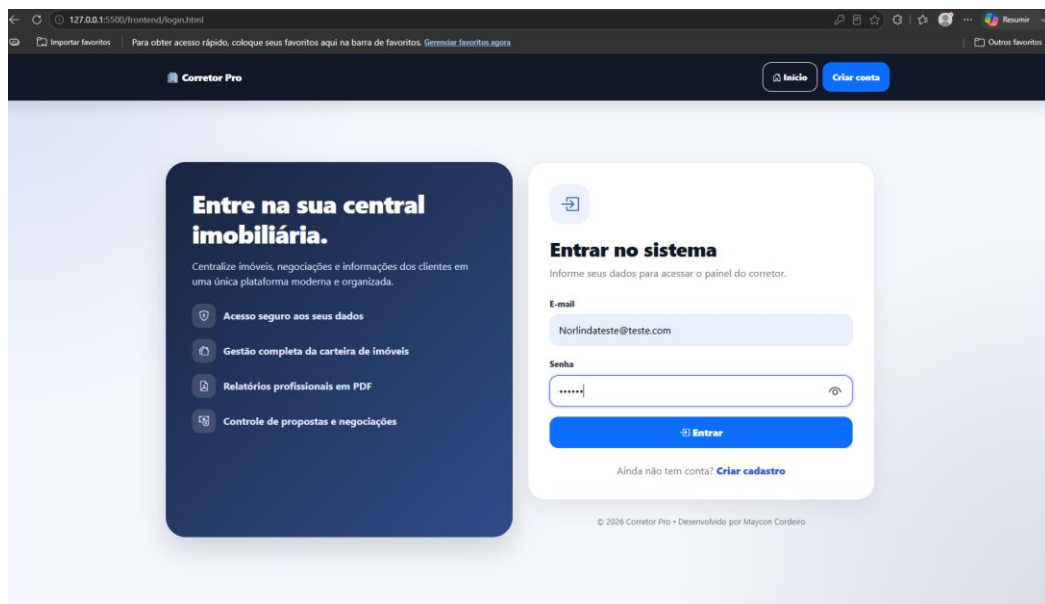


Imagem 4

- Cadastro

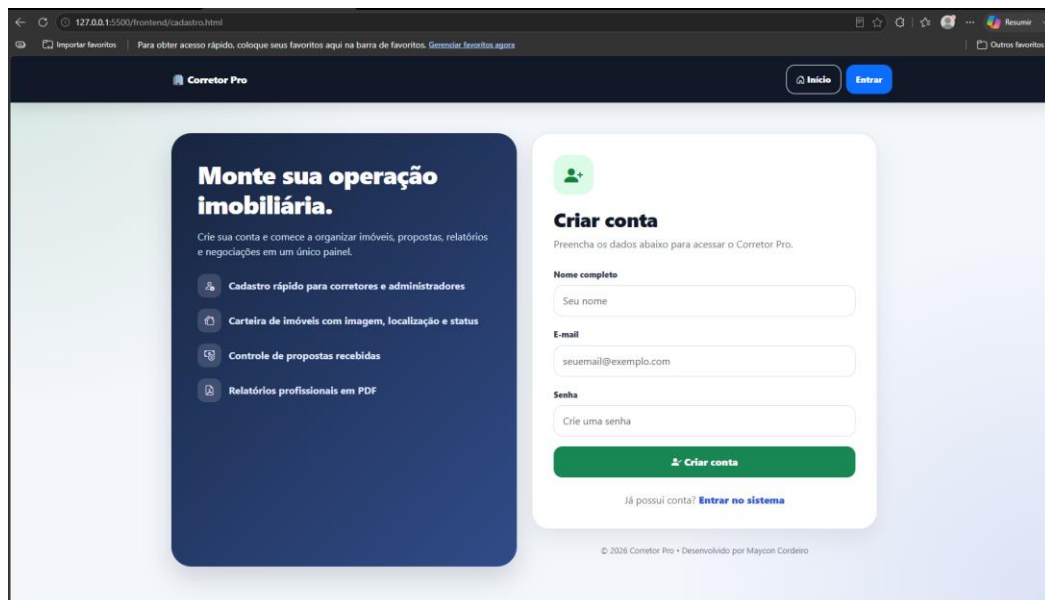


Imagem 5

- Dashboard

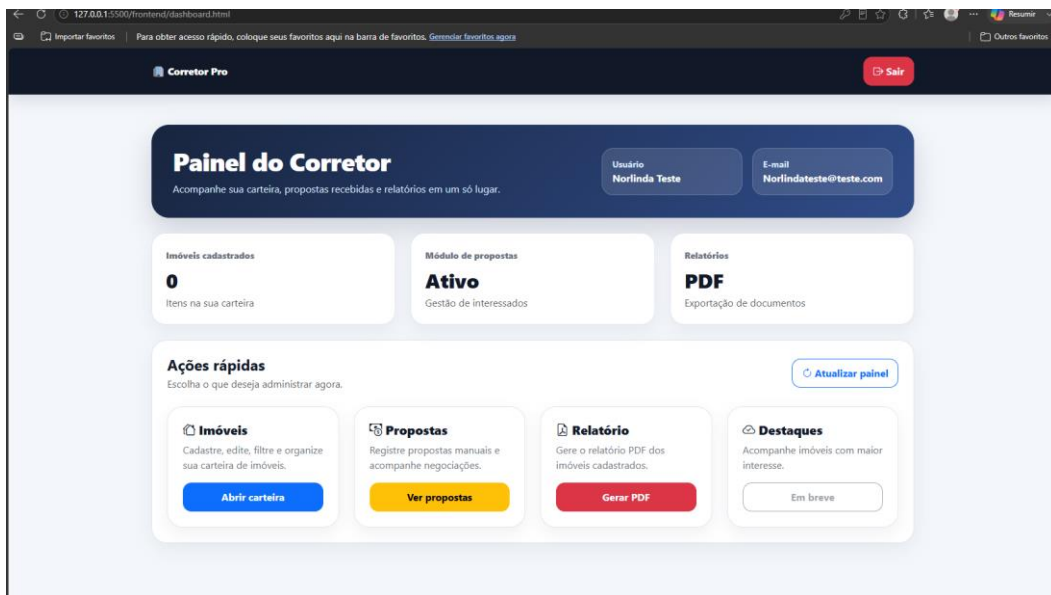


Imagem 6

- Imóveis

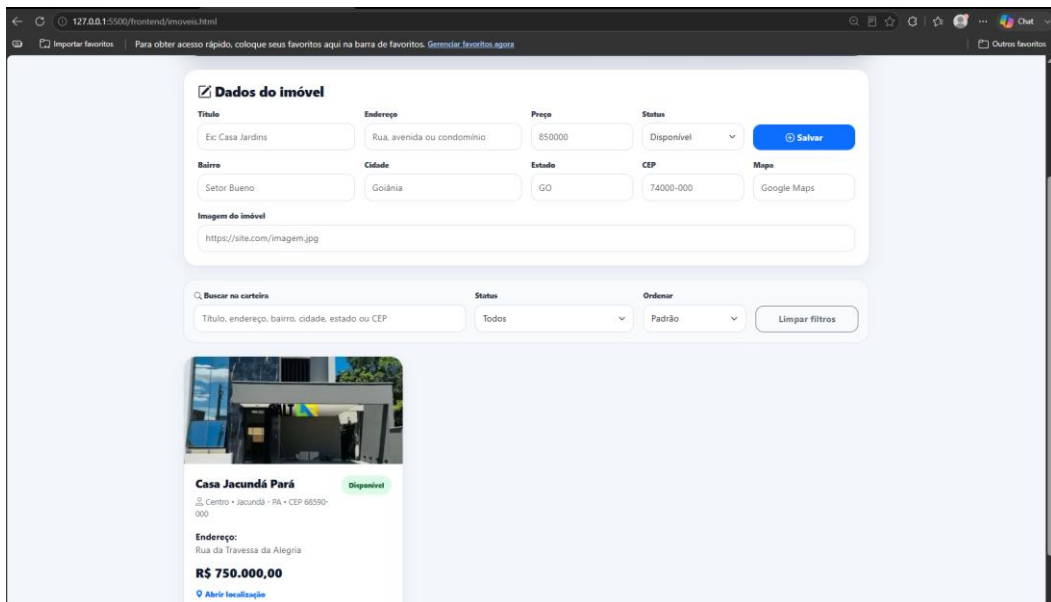


Imagem 7

- Propostas

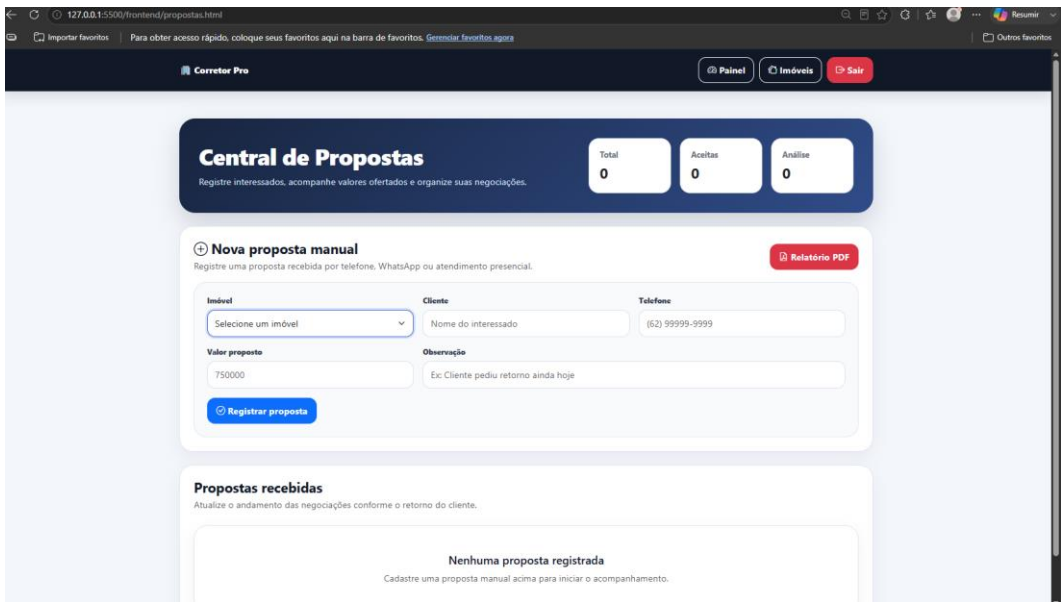


Imagem 8

- Relatório PDF

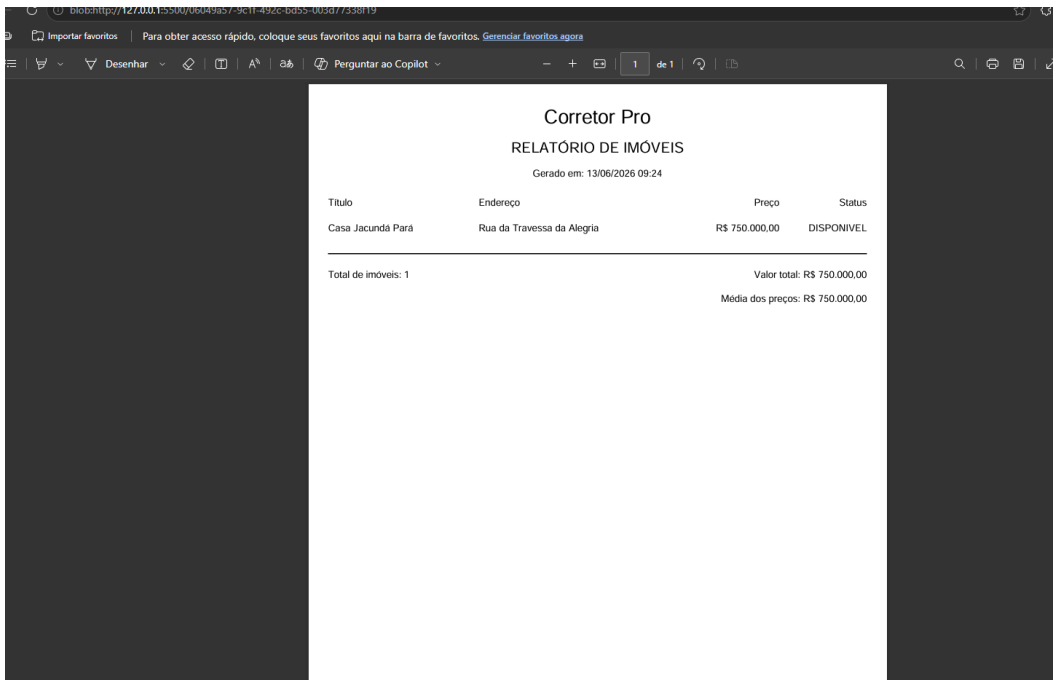


Imagem 9

Corretor Pro

RELATÓRIO DE PROPOSTAS

Gerado em: 13/06/2025 09:26

Cliente	Telefone	Imóvel	Endereço	Valor	Status	Data
Osório Jacometo	6299996669	Casa Jacundá Para	Rua da Travessa da Alegria	R\$ 690.000,00	Em análise	13/06/2025

Total de propostas: 1 Em análise: 1 | Aceitas: 0 | Recusadas: 0 Valor total: R\$ 690.000,00
 Média das propostas: R\$ 690.000,00

Desenvolvido por Maycon Cordeiro

Página 1

Imagens 10

- Banco PostgreSQL

Object Explorer

Servers (2)

PostgreSQL 18

Databases (2)

corretor_db

Casts

Catalogs

Event Triggers

Extensions

Foreign Data Wrappers

Languages

Publications

Schemas (1)

public

Aggregates

Collations

Domains

FTS Configurations

FTS Dictionaries

FTS Parsers

FTS Templates

Foreign Tables

Functions

Materialized Views

Operators

Procedures

Sequences

Tables (3)

imovel

proposta

usuario

Columns

Constraints

Indexes

RLS Policies

Rules

Triggers

Trigger Functions

Types

Query

Query History

1 SELECT table_name

2 FROM information_schema.tables

3 WHERE table_schema = 'public';

Scratch Pad

Data Output

Messages

Notifications

Showing rows: 1 to 3

Page No: 1 of 1

table_name
usuario
proposta
imovel

Total rows: 3 Query complete 00:00:00.063

CRLF Ln 3, Col 31

Imagem 11

- Vercel

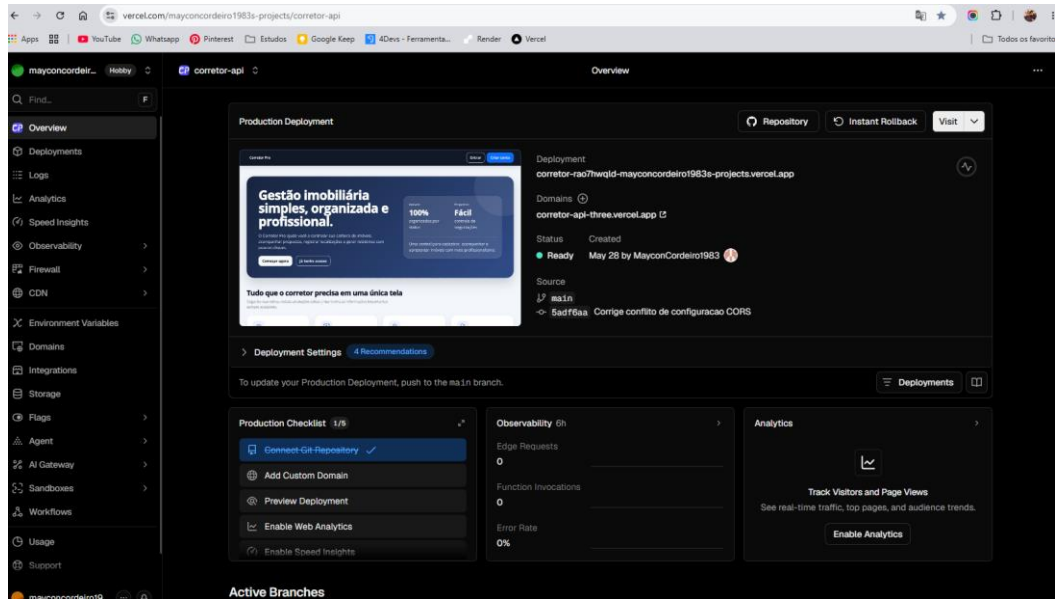


Imagem 12

- GitHub

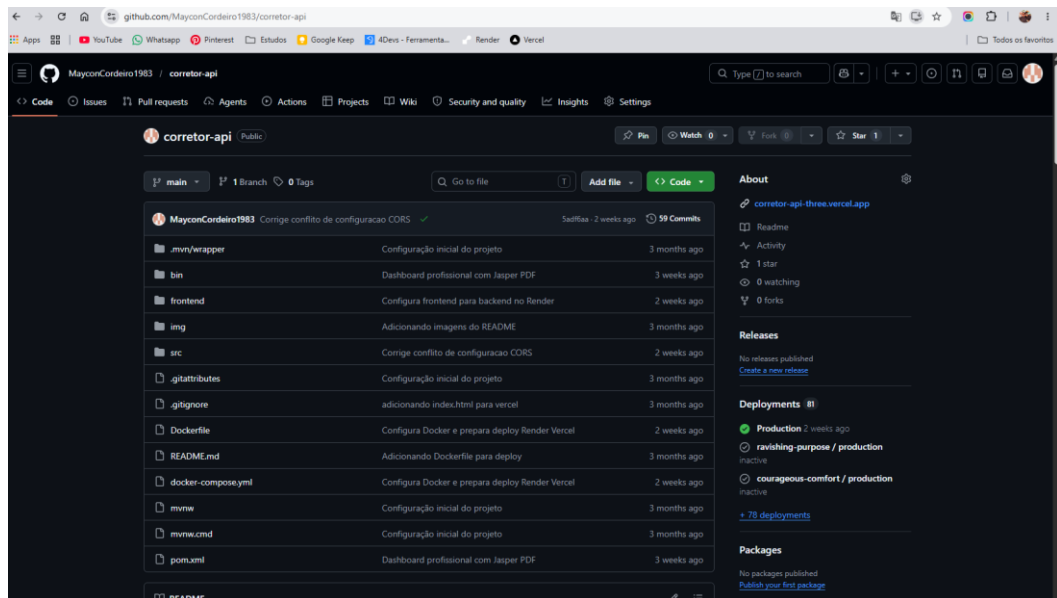


Imagem 13

20. CONCLUSÃO

O projeto Corretor Pro atingiu todos os objetivos propostos, permitindo o gerenciamento de imóveis e propostas através de uma aplicação web moderna, segura e funcional.

Foram aplicados os conceitos estudados durante o curso, incluindo Spring Boot, Spring Security, JPA, Hibernate, AJAX e JasperReports.

Além disso, o sistema foi preparado para execução em ambientes de nuvem utilizando Docker, Render e Vercel, aproximando o projeto das práticas adotadas pelo mercado profissional.

O desenvolvimento desta aplicação proporcionou experiência prática em diversas áreas da engenharia de software, contribuindo significativamente para a formação acadêmica e profissional.

21. REFERÊNCIAS

- Spring Boot Documentation. VMware, 2026.
- Spring Security Documentation. VMware, 2026.
- PostgreSQL Official Documentation. PostgreSQL Global Development Group, 2026.
- Hibernate ORM Documentation. Red Hat, 2026.
- Docker Documentation. Docker Inc., 2026.
- Render Documentation. Render Inc., 2026.
- Vercel Documentation. Vercel Inc., 2026.

APÊNDICE A – DIAGRAMAS UML COMPLETOS

Este apêndice complementa as seções 7 e 8 da documentação, apresentando os diagramas UML separados por módulos do sistema. A separação facilita a compreensão das funcionalidades e das classes implementadas no projeto Corretor Pro.

A.1 Diagramas de Casos de Uso

A.1.1 Diagrama Geral de Casos de Uso

Representa a visão geral das interações do corretor com o sistema, contemplando cadastro, login, dashboard, imóveis, propostas e relatórios.

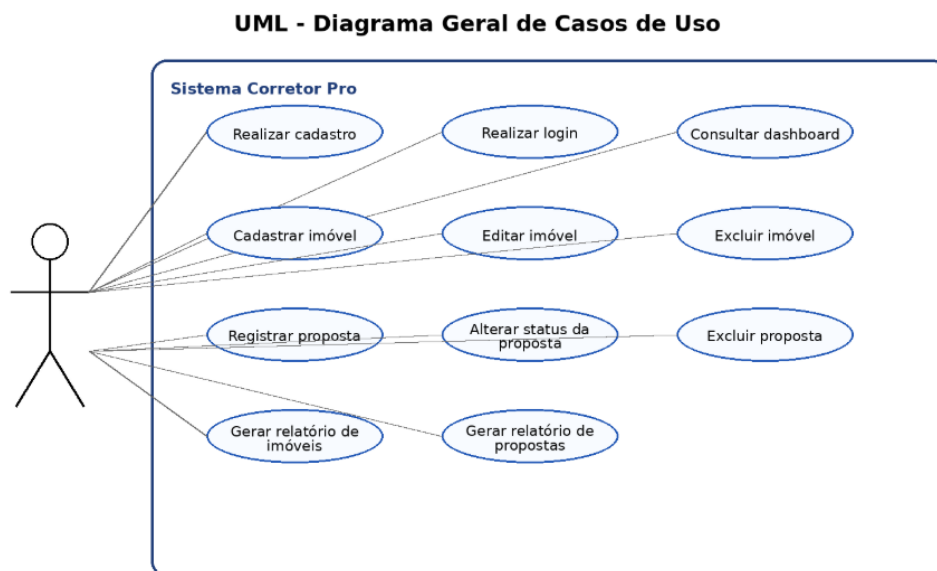


Figura UML 1 – Diagrama Geral de Casos de Uso.

A.1.2 Diagrama de Casos de Uso – Autenticação e Acesso

Detalha as funcionalidades de acesso ao sistema, incluindo criação de conta, login, validação de credenciais, geração de token e encerramento de sessão.

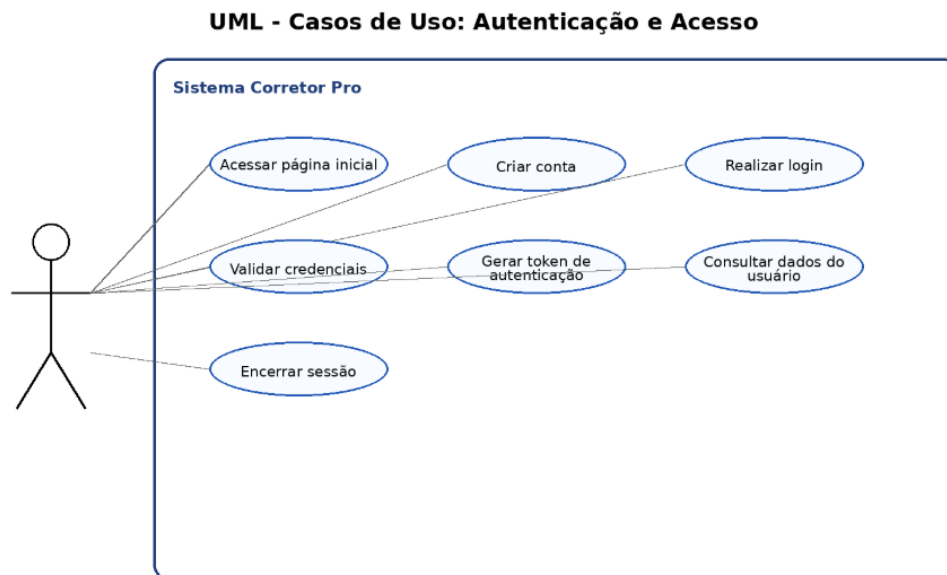


Figura UML 2 – Diagrama de Casos de Uso – Autenticação e Acesso.

A.1.3 Diagrama de Casos de Uso – Gestão de Imóveis

Demonstra as funcionalidades relacionadas à carteira de imóveis, como cadastrar, consultar, filtrar, ordenar, editar, excluir e gerar relatório de imóveis.

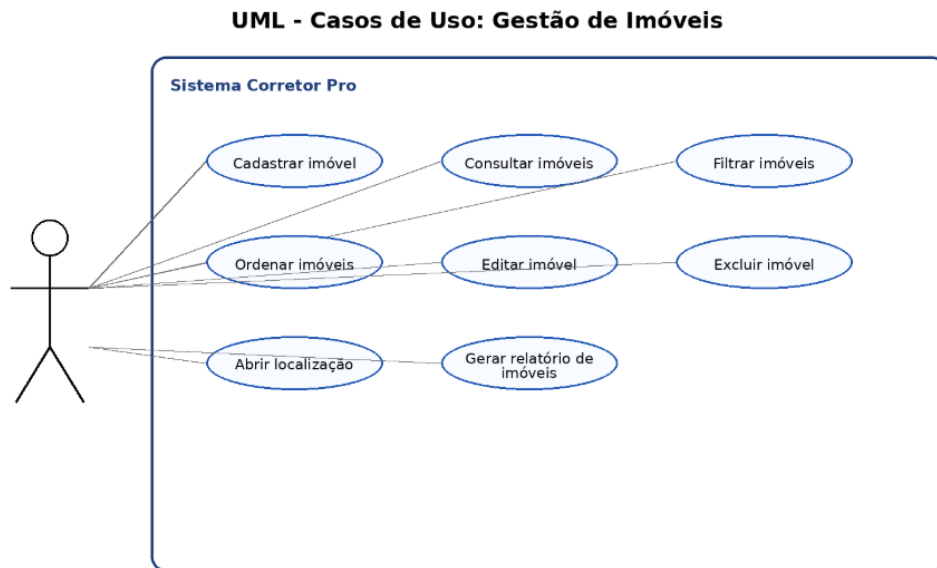


Figura UML 3 – Diagrama de Casos de Uso – Gestão de Imóveis.

A.1.4 Diagrama de Casos de Uso – Gestão de Propostas

Apresenta as ações relacionadas às propostas recebidas, incluindo registro manual, consulta, aceite, recusa, exclusão e geração de relatório.



Figura UML 4 – Diagrama de Casos de Uso – Gestão de Propostas.

A.1.5 Diagrama de Casos de Uso – Relatórios e Persistência

Mostra o fluxo de solicitação dos relatórios PDF, busca de dados, compilação dos arquivos JRXML e consulta ao banco PostgreSQL.

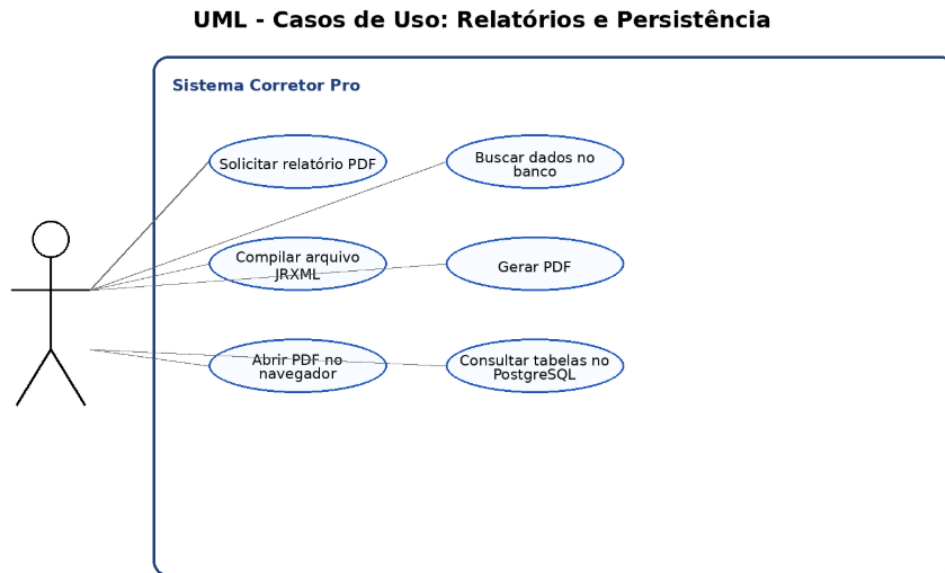


Figura UML 5 – Diagrama de Casos de Uso – Relatórios e Persistência.

A.2 Diagramas de Classes

A.2.1 Diagrama de Classes – Domínio

Representa as entidades centrais do domínio: Usuario, Imovel, Proposta e os enumeradores StatusImovel e StatusProposta. O relacionamento indica que um usuário pode possuir vários imóveis e um imóvel pode receber várias propostas.

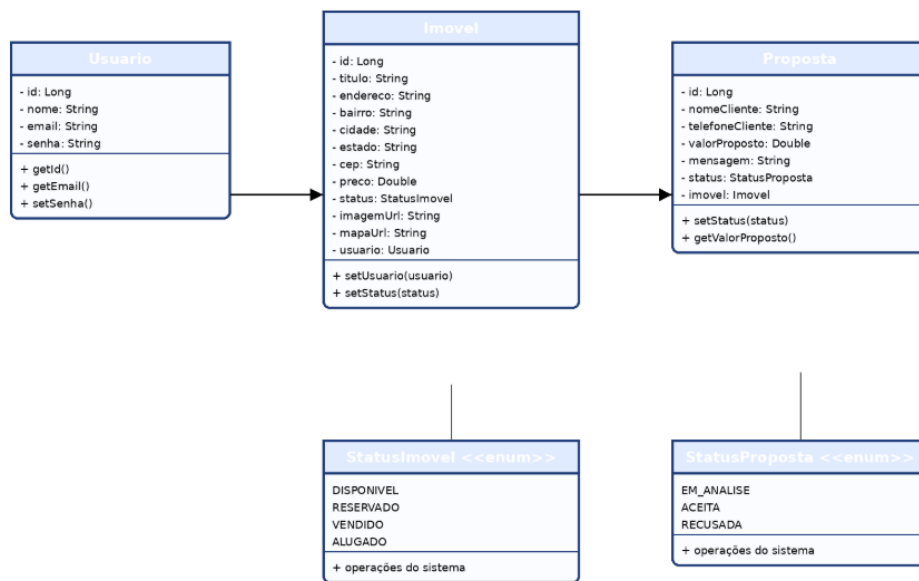


Figura UML C1 – Diagrama de Classes – Domínio.

A.2.2 Diagrama de Classes – Camadas MVC/REST

Demonstra a organização em camadas do backend: Controller, Service e Repository. Essa estrutura separa responsabilidades entre recebimento das requisições, regras de negócio e persistência dos dados.

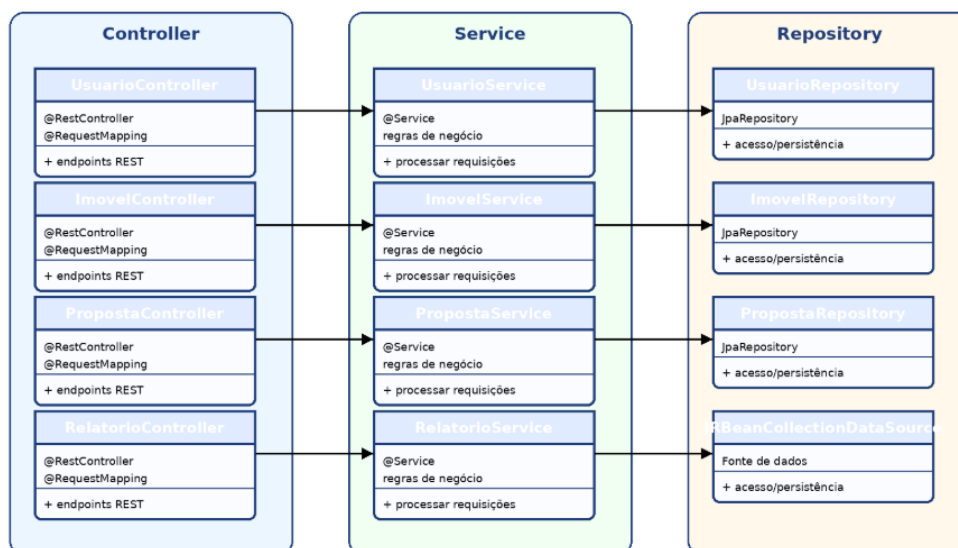


Figura UML C2 – Diagrama de Classes – Camadas MVC/REST.

A.2.3 Diagrama de Classes – Segurança e Relatórios

Apresenta as classes relacionadas à segurança da aplicação e à geração de relatórios: SecurityConfig, JwtAuthFilter, JwtUtil, RelatorioController e RelatorioService.

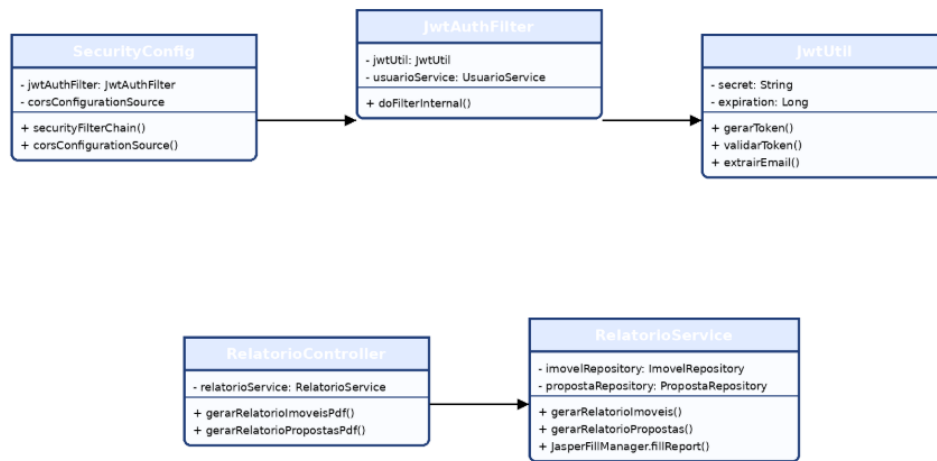


Figura UML C3 – Diagrama de Classes – Segurança e Relatórios.

APÊNDICE B – DESCRIÇÃO DAS IMAGENS DO SISTEMA

As descrições a seguir explicam as imagens inseridas na seção de demonstração do sistema, indicando a finalidade de cada tela e sua relação com os requisitos do projeto.

Imagem 1 – Entidade Usuario com JPA/Hibernate

A imagem apresenta a classe **Usuario** mapeada através das anotações JPA/Hibernate. A utilização da anotação `@Entity` permite que a classe seja convertida automaticamente em uma tabela do banco de dados PostgreSQL. Também são utilizadas anotações para definição da chave primária, geração automática de identificadores e restrições dos atributos, demonstrando o mapeamento objeto-relacional implementado no sistema Corretor Pro.

Imagem 2 – Repositório UsuarioRepository

A imagem demonstra a interface **UsuarioRepository**, responsável pela camada de persistência da entidade Usuario. O repositório herda de `JpaRepository`, disponibilizando operações de cadastro, consulta, atualização e exclusão de registros sem a necessidade de implementação manual de comandos SQL. Essa abordagem reduz a complexidade do código e facilita a comunicação entre a aplicação Spring Boot e o banco de dados PostgreSQL.

Imagem 3 – Tela Inicial

Apresenta a página inicial do Corretor Pro, com a proposta do sistema e atalhos para login e criação de conta. Também destaca os módulos principais: carteira de imóveis, propostas, localização e relatórios.

Imagem 4 – Tela de Login

Demonstra a autenticação do usuário por e-mail e senha. Após a validação pelo backend, o usuário recebe autorização para acessar o painel do corretor.

Imagem 5 – Tela de Cadastro

Permite o registro de novos usuários no sistema, solicitando nome completo, e-mail e senha. Os dados são enviados à API e persistidos no PostgreSQL.

Imagem 6 – Dashboard

Exibe o painel principal do usuário autenticado, com informações do corretor, total de imóveis cadastrados, módulo de propostas e acesso à geração de relatórios.

Imagem 7 – Tela de Imóveis

Apresenta o formulário de cadastro e edição de imóveis, contendo campos como título, endereço, preço, status, bairro, cidade, estado, CEP, imagem e mapa.

Imagem 8 – Carteira de Imóveis

Lista os imóveis cadastrados em formato visual, exibindo imagem, localização, preço, status e ações disponíveis para gerenciamento.

Imagem 9 – Tela de Propostas

Permite registrar propostas manuais para imóveis cadastrados, informando cliente, telefone, valor proposto e observação. Também exibe as propostas recebidas e seus status.

Imagem 10 – Relatório de Imóveis

Demonstra o PDF gerado com JasperReports contendo os dados dos imóveis cadastrados, valor total, média de preços e total de registros.

Imagem 11 – Relatório de Propostas

Demonstra o PDF gerado com JasperReports contendo cliente, telefone, imóvel, endereço, valor, status, data e resumo financeiro das propostas.

Imagem 12 – Banco PostgreSQL

Mostra o pgAdmin com as tabelas principais do sistema: usuario, imovel e proposta, comprovando a persistência dos dados com JPA e Hibernate.

Imagem 13 – Vercel

Evidencia o ambiente de hospedagem do frontend, responsável por disponibilizar a interface web ao usuário.

Imagem 14 – GitHub

Mostra o repositório com o código-fonte do projeto, incluindo backend, frontend, Dockerfile, docker-compose.yml e demais arquivos necessários para a entrega.